

Documentación técnica - Gestor Escuela de Teatro

Aplicación web PHP/MySQL para la gestión de asistencia de una escuela de teatro

Campo	Valor
Proyecto	Gestor-Escuela-de-Teatro
Tipo de sistema	Aplicación web monolítica con páginas renderizadas en PHP
Entorno objetivo	Desarrollo local con XAMPP, Apache, PHP y MySQL
Documento	Documentación técnica de apoyo al portfolio
Curso de referencia	2025-2026

Proyecto académico desarrollado en equipo · Versión de portfolio revisada por Lucía Jiménez Travé

Nota de alcance: el documento describe el estado actual del proyecto. Los apartados de la plantilla que no aplican al repositorio se indican como No aplica o No implementado para evitar documentar infraestructura o APIs inexistentes.

1. Introducción

1.1 Objetivo del documento

El objetivo de esta documentación es explicar la estructura técnica y funcional del proyecto para que desarrolladores, QA, soporte y personas evaluadoras puedan entender cómo se instala, cómo funciona y cómo se mantiene la aplicación.

- Alcance técnico: arquitectura PHP, rutas, controladores, vistas, base de datos, seguridad básica, instalación y scripts SQL.
- Alcance funcional: gestión de usuarios por rol, alumnado, grupos, clases, asistencia, tokens de recuperación, clases de prueba y eventos.
- Público objetivo: equipo de desarrollo, responsables de pruebas, soporte funcional y cliente académico.

1.2 Descripción general del proyecto

La aplicación resuelve la gestión diaria de una escuela de teatro: organización de alumnos posibles y matriculados, asignación a grupos, programación de clases, control de asistencia, avisos de ausencia, generación de tokens de recuperación y reserva de clases recuperables.

El sistema se organiza por roles. El administrador gestiona alumnos, grupos, salas y eventos; el profesor trabaja con sus grupos, clases, asistencia y observaciones; el alumno matriculado consulta próximas clases, calendario, tokens y recuperaciones; el alumno en estado posible actúa como invitado y puede reservar una clase de prueba si tiene token disponible.

La arquitectura es cliente-servidor tradicional. El navegador solicita páginas PHP al servidor Apache; el front controller `index.php` carga configuración y rutas; los controladores consultan MySQL con `mysqli`; las vistas PHP devuelven HTML, CSS y JavaScript al cliente.

1.3 Alcance

Incluido	Excluido o no implementado	Limitaciones conocidas
Login, sesiones y redirección por rol.	Registro público autónomo de usuarios.	Las credenciales de BD están fijas en `config/database.php` para XAMPP.
CRUD operativo de alumnos, grupos, salas y eventos.	API REST/GraphQL externa.	No hay versionado de API porque el sistema no expone API JSON.
Control de asistencia, ausencias y tokens.	Pasarela de pago, correo transaccional o webhooks.	La lógica de pagos existe como tabla `bloque_pago`, pero no es el flujo principal documentado.
Scripts de instalación, semillas y datos demo.	Docker, cloud, CI/CD y despliegue productivo.	El entorno documentado es desarrollo local.
CSRF en formularios, consultas preparadas y escape de salida.	MFA, OAuth, JWT, rate limiting y hardening productivo completo.	La seguridad depende también de configurar correctamente Apache/PHP en el entorno real.

1.4 Glosario y acrónimos

Término	Significado
Alumno posible / invitado	Persona interesada que puede reservar una clase de prueba.
Alumno matriculado	Alumno activo inscrito en uno o más grupos.
Grupo	Conjunto de clases recurrentes con profesor, sala, horario, nivel, tipo y curso.
Clase	Sesión concreta de un grupo en una fecha y hora.
Asistencia	Registro por alumno y clase con estado `asiste`, `ausente` o `avisado`.
Token	Crédito usado para reservar una recuperación o una clase de prueba.
Recuperación	Reserva de una clase alternativa usando un token disponible.

1.5 Referencias

- Repositorio público: <https://github.com/LucyTrave/Gestor-Escuela-de-Teatro>
- `README.md` como guía rápida de instalación, usuarios demo, rutas y modelo de datos.
- `setup.php` como fuente principal del esquema inicial de base de datos.
- `routes/web.php` como mapa de rutas HTTP.
- Controladores en `app/controllers` y vistas en `app/views`.
- Scripts SQL en `sql/` y migraciones en `database/migrations/`.

2. Arquitectura del Sistema

2.1 Visión general de arquitectura

La aplicación sigue una arquitectura monolítica PHP con separación práctica entre rutas, controladores y vistas. No utiliza Composer ni framework; las dependencias se resuelven con `require\_once` y el enrutado se implementa con condicionales sobre la URI y el método HTTP.

```
Navegador
-> Apache / XAMPP
-> index.php define ROOT y BASE_URL
-> config/database.php crea la conexión mysqli
-> routes/web.php selecciona controlador y acción
-> app/controllers/*.php ejecutan lógica de negocio
-> MySQL punto_de_partida
<- app/views/*.php renderizan HTML
<- HTML + CSS + JavaScript
```

2.2 Diagrama de arquitectura

Capa	Elemento	Responsabilidad
Cliente web	Navegador	Envía solicitudes GET/POST, muestra HTML y ejecuta JavaScript de confirmaciones y calendarios.
Servidor web	Apache en XAMPP	Sirve `index.php`, archivos públicos y ejecuta PHP.
Entrada	`index.php`	Define constantes, carga la conexión de base de datos y delega en el enrutador.
Enrutado	`routes/web.php`	Relaciona URI y método HTTP con el controlador correspondiente.
Backend	`app/controllers`	Valida sesión/rol, procesa formularios, consulta o modifica MySQL y selecciona la vista.
Presentación	`app/views`	Plantillas PHP por rol: admin, profesor, alumno, invitado, auth y dashboard.
Persistencia	MySQL `punto_de_partida`	Guarda usuarios, grupos, clases, asistencia, tokens, recuperaciones, eventos y salas.
Estáticos	`public/css`, `public/js`	Define estilos globales y JavaScript complementario.
Servicios externos	CDN SweetAlert2 y Font Awesome	Mejora visual de confirmaciones e iconos en algunas vistas.

2.3 Tecnologías utilizadas

Área	Tecnología	Uso en el proyecto
Frontend	HTML renderizado por PHP	Pantallas por rol y formularios tradicionales.
Frontend	CSS propio (`public/css/styles.css`)	Diseño visual, dashboards, tablas, formularios y calendarios.
Frontend	JavaScript propio e inline	Confirmaciones, modales, filtros visuales y pequeñas interacciones.
Frontend	SweetAlert2 por CDN	Alertas y confirmaciones enriquecidas en vistas de

		alumno, invitado, profesor y admin.
Frontend	Font Awesome por CDN	Iconografía en vistas del profesor.
Backend	PHP	Controladores, sesiones, renderizado de vistas y acceso a datos.
Backend	`mysql`	Conexión y consultas preparadas contra MySQL.
Autenticación	Sesiones PHP + `password_hash`/`password_verify`	Login, persistencia de usuario autenticado y verificación de contraseña.
Base de datos	MySQL / InnoDB / utf8mb4	Persistencia relacional con claves primarias, foráneas y restricciones únicas.
Infraestructura	XAMPP	Entorno local con Apache, PHP y MySQL.

3. Requisitos del Sistema

3.1 Requisitos funcionales

Código	Requisito	Implementación
RF-01	Autenticación de usuarios.	`AuthController` valida email y contraseña contra `usuario`.
RF-02	Redirección según rol.	Admin a `/admin`, profesor a `/profesor`, alumno posible a `/invitado`, alumno matriculado a `/alumno`.
RF-03	Gestión de alumnos posibles.	Listado, filtros, creación, edición, eliminación lógica y matriculación.
RF-04	Gestión de alumnos matriculados.	Listado, filtros, detalle, edición, baja y asignación a grupos.
RF-05	Gestión de grupos y salas.	CRUD funcional desde administración y gestión parcial desde profesor.
RF-06	Gestión de clases.	Creación, edición, listado, calendario y control de duplicidades/solapes.
RF-07	Control de asistencia.	Estados por alumno y clase; registro único mediante `uk_asistencia`.
RF-08	Aviso de ausencia.	Alumno puede avisar; si cumple antelación y máximo, obtiene token.
RF-09	Recuperación de clases.	Alumno usa tokens para reservar clases recuperables con plazas.
RF-10	Clase de prueba.	Alumno posible reserva/cancela una clase de prueba usando token.
RF-11	Eventos especiales.	Admin crea eventos, inscribe alumnos, controla aforo y elimina inscripciones.
RF-12	Observaciones de profesor.	Profesor añade observaciones a alumnos, grupos o clases.

3.2 Requisitos no funcionales

Categoría	Criterio
Rendimiento	Adecuado en entorno local: las consultas se ejecutan sobre MySQL con índices y claves básicas.
Escalabilidad	Escalabilidad limitada por arquitectura monolítica y entorno XAMPP.
Seguridad	Sesiones PHP, contraseñas hash, CSRF en formularios sensibles, consultas preparadas y escape de salida.
Disponibilidad	Depende de Apache/MySQL en XAMPP.
Mantenibilidad	Estructura simple por carpetas y roles; README y esta documentación sirven como guía.
Compatibilidad	Diseñado para navegadores modernos y PHP con extensión `mysqli`.

3.3 Requisitos de hardware y software

- Servidor de desarrollo: equipo local con XAMPP instalado.
- Software mínimo: Apache, MySQL/MariaDB compatible, PHP con extensión `mysqli`.
- Ubicación recomendada: carpeta del proyecto dentro de `htdocs`.
- Navegadores compatibles: Chrome, Firefox, Edge o Safari recientes.
- Base de datos inicial: creada por `setup.php` con nombre `punto\_de\_partida`.

4. Diseño del Frontend

4.1 Arquitectura frontend

El frontend son vistas PHP renderizadas en servidor. Cada petición devuelve una página HTML completa y los formularios envían datos mediante POST. La navegación se organiza por rutas y roles.

4.2 Estructura de carpetas

```
app/views/  
  auth/login.php  
  admin/*.php  
  profesor/*.php  
  alumno/*.php  
  invitado/*.php  
  dashboard/index.php  
public/css/styles.css
```

4.3 Componentes principales

Componente	Vistas	Descripción
Login	`app/views/auth/login.php`	Formulario de acceso por email y contraseña.
Layout admin	`app/views/admin/partials`	Cabecera, navegación y pie para pantallas administrativas.
Panel profesor	`app/views/profesor/_header.php`, `_footer.php`	Cabecera, navegación, mensajes y confirmaciones comunes.
Dashboards	Admin, profesor, alumno e invitado	Tarjetas de resumen, accesos rápidos y próximas acciones.
Tablas	Admin y profesor	Listados filtrables/editables de alumnos, grupos, clases, salas y eventos.
Calendarios	Alumno, invitado y profesor	Vista mensual y selección de clases o eventos por fecha.
Formularios	Todas las áreas	Altas, ediciones, matriculación, asistencia, ausencias y recuperaciones.
Alertas	SweetAlert2 + avisos PHP	Confirmaciones visuales, mensajes de éxito y advertencias.

4.4 Manejo de errores

- Errores de login redirigen a `/login?error=1`.
- Acciones POST inválidas o con CSRF incorrecto redirigen con mensajes como `error\_datos`.
- Las vistas muestran avisos a partir de parámetros `mensaje`, `ok` o indicadores similares.
- No hay error boundaries porque no se usa React ni framework SPA.
- La ruta no reconocida devuelve código HTTP 404 y el mensaje `404 - Página no encontrada`.

4.5 Seguridad en frontend

- Las salidas dinámicas se imprimen mayoritariamente con `htmlspecialchars`.
- Los formularios sensibles incluyen campo oculto CSRF mediante `Csrf::field()`.
- Las confirmaciones JavaScript reducen acciones accidentales.
- No se almacena estado sensible en `localStorage` ni `sessionStorage`.

5. Diseño del Backend

5.1 Arquitectura backend

El backend está escrito en PHP sin framework. Se comporta como una arquitectura MVC ligera: `routes/web.php` decide el controlador, los controladores aplican reglas de negocio y las vistas renderizan la respuesta. No existe capa de servicios formal ni repositorios separados; la lógica SQL vive principalmente en controladores y `setup.php`.

5.2 Estructura del proyecto

```
app/  
  controllers/  
    AuthController.php  
    AdminController.php  
    ProfesorController.php  
    AlumnoController.php  
    InvitadoController.php  
  helpers/Csrf.php  
  models/ejemplo.php  
  views/  
config/database.php  
routes/web.php  
index.php  
setup.php  
sql/  
database/migrations/
```

5.3 Rutas HTTP

Área	Rutas	Descripción
Auth	`/`, `/login`, `/logout`, `/dashboard`	Entrada, login, cierre de sesión y redirección por rol.
Admin	`/admin`, `/admin/posibles`, `/admin/matriculados`	Panel y gestión de alumnos posibles/matriculados.
Admin	`/admin/grupos`, `/admin/grupos/crear`, `/admin/grupos/editar`, `/admin/grupos/alumnos`	Gestión de grupos, alumnos de grupo y clases generadas.
Admin	`/admin/salas/crear`, `/admin/salas/actualizar`, `/admin/salas/eliminar`	Gestión de salas.
Admin	`/admin/especiales`, `/admin/especiales/gestionar`, `/admin/especiales/crear`	Eventos especiales e inscripción de alumnos.
Profesor	`/profesor`, `/profesor/alumnos`, `/profesor/grupos`, `/profesor/clases`, `/profesor/asistencia`, `/profesor/calendario`	Trabajo diario del profesor con alumnos, grupos, clases, calendario y asistencia.
Alumno	`/alumno`, `/alumno/tokens`, `/alumno/recuperar`, `/alumno/calendario`	Panel, tokens, recuperación y calendario.
Alumno POST	`/alumno/avisar-ausencia`, `/alumno/anular`	Acciones de ausencia y recuperación

	ausencia`, `/alumno/recuperar-clase`, `/alumno/anular-recuperacion`	protegidas por CSRF.
Invitado	`/invitado`, `/invitado/calendario`, `/invitado/reservar`, `/invitado/cancelar`	Reserva y cancelación de clase de prueba para alumnos posibles.

API REST / GraphQL: No aplica. Las rutas devuelven páginas HTML o redirecciones; no hay versionado `/api/v1`, contrato JSON ni códigos HTTP de API.

5.4 Autenticación y autorización

- `AuthController::processLogin()` busca el usuario por email y verifica contraseña con `password\_verify`.
- Las contraseñas demo se guardan hasheadas con `password\_hash` desde `setup.php`.
- La sesión guarda `usuario\_id`, `usuario\_nombre`, `usuario\_email`, `usuario\_rol` y, para alumnos, `alumno\_id` y `alumno\_estado`.
- Cada controlador protegido valida rol y redirige al panel correspondiente si el acceso no corresponde.
- JWT, OAuth2 y MFA: No implementado.

5.5 Validaciones

- Validación de presencia para campos críticos como email, contraseña, identificadores y formularios.
- Conversión de identificadores a entero con `(int)` antes de usarlos en consultas.
- Validación de fechas de calendario con `DateTime::createFromFormat`.
- Validación de pertenencia: alumnos solo actúan sobre sus clases/tokens; profesores sobre sus grupos salvo acceso admin.
- No existen DTOs formales; las validaciones están implementadas en controladores.

5.6 Manejo de errores

- El patrón principal es redirigir con un parámetro de mensaje y mostrarlo en la vista.
- Errores de base de datos críticos en conexión usan `die()` con mensaje de error.
- Algunas operaciones usan excepciones de MySQL para evitar eliminaciones inválidas.
- No hay sistema centralizado de logs ni respuesta JSON estándar.

5.7 Integraciones externas

Integración	Estado
SweetAlert2 CDN	Usado para alertas y confirmaciones en varias vistas.
Font Awesome CDN	Usado para iconos del área de profesor.

Pasarela de pago	No implementado.
Correo electrónico	No implementado.
Webhooks/APIs de terceros	No implementado.

6. Base de Datos

6.1 Modelo de datos

La base de datos se llama `punto\_de\_partida` y se crea desde `setup.php` con InnoDB y `utf8mb4`. Las tablas principales modelan usuarios, roles, alumnos, grupos, clases, asistencia, tokens, recuperaciones, eventos y salas.

6.2 Diagrama entidad-relación

Relación	Cardinalidad	Descripción
usuario -> admin/profesor/alumno	1 a 0..1	El usuario base contiene credenciales y rol; las tablas específicas guardan datos de perfil.
profesor -> grupo	1 a N	Cada grupo tiene un profesor asignado.
sala -> grupo/clase/evento_grupal	1 a N	Las salas se usan para grupos, clases concretas y eventos.
alumno <-> grupo	N a M	`alumno_grupo` permite inscripciones activas por grupo.
grupo -> clase	1 a N	Cada grupo genera clases concretas con fecha y hora.
alumno <-> clase	N a M	`asistencia` registra el estado del alumno en una clase.
asistencia -> token	1 a 0..1/N	Un aviso válido puede generar token vinculado a la asistencia origen.
token -> recuperacion	1 a N lógico	Una recuperación consume un token marcándolo como usado.
alumno <-> evento_grupal	N a M	`inscripcion_evento` registra alumnos inscritos en eventos.
profesor -> observacion_profesor	1 a N	Observaciones sobre alumnos, grupos o clases.

6.3 Diccionario de datos

Tabla	Campos clave	Uso principal
usuario	`id`, `email`, `password_hash`, `rol`	Credenciales, rol y login.
admin	`usuario_id`, `nombre`, `apellidos`	Perfil del administrador.
profesor	`usuario_id`, `nombre`, `apellidos`, `telefono`	Perfil de profesorado.
alumno	`id`, `usuario_id`, `estado`, `nivel`, fechas de interés	Alumnos posibles, matriculados o de baja.
horario_posible	`alumno_id`, `dia_semana`,	Disponibilidad de alumnos posibles.

	`tramo_horario`	
sala	`id`, `nombre`, `espacio_nombre`, `direccion`, `aforo_maximo`	Espacios físicos donde se imparten clases o eventos.
grupo	`profesor_id`, `sala_id`, `nombre`, `horario`, `nivel`, `tipo`, `curso`, `activo`	Oferta recurrente de clases.
alumno_grupo	`alumno_id`, `grupo_id`, `activo`	Inscripción de alumnos en grupos; restricción única alumno/grupo.
clase	`grupo_id`, `sala_id`, `fecha`, horas, `cupos_maximo`, `estado`	Sesiones concretas programadas.
asistencia	`alumno_id`, `clase_id`, `estado`, `fecha_aviso`, `aviso_valido`	Estado de asistencia; única por alumno/clase.
token	`alumno_id`, `tipo`, `asistencia_origen_id`, `fecha_caducidad`, `usado`	Crédito para recuperación o prueba.
recuperacion	`alumno_id`, `token_id`, `clase_origen_id`, `clase_recuperacion_id`, `estado`	Reserva de clase alternativa.
bloque_pago	`alumno_id`, `grupo_id`, `importe`, `pagado`	Estructura preparada para bloques de pago.
evento_grupal	`nombre`, `tipo`, `fecha`, `plazas_maximas`, `sala_id`, `hora`	Intensivos o salidas de teatro.
inscripcion_evento	`alumno_id`, `evento_id`, `estado`	Inscripción única de alumnos en eventos.
observacion_profesor	`profesor_id`, `tipo_destino`, `destino_id`, `texto`	Notas del profesor sobre alumnos, grupos o clases.

6.4 Seguridad de datos

- Contraseñas almacenadas como hash, nunca en texto plano.
- Conexión configurada con `utf8mb4` para soportar caracteres españoles.
- Claves foráneas InnoDB y restricciones únicas para evitar duplicidades relevantes.

7. Comunicación Cliente-Servidor

7.1 Flujo de comunicación

- GET de página: navegador solicita ruta, `routes/web.php` instancia controlador, controlador consulta datos y carga vista PHP.
- POST de formulario: navegador envía datos y `csrf\_token`; controlador valida sesión/rol/CSRF, ejecuta operación y redirige.
- Mensajes al usuario: se transportan normalmente por query string (`mensaje`, `ok`, `error`) y se renderizan en la vista.

7.2 Formato de mensajes

El formato principal es HTML generado en servidor. Los formularios usan `application/x-www-form-urlencoded` o el formato estándar de POST de navegador.

7.3 Manejo de sesiones

- La sesión se inicia con `session\_start()` en controladores que necesitan autenticación.
- El estado autenticado se guarda en `\$\_SESSION`.
- El cierre de sesión limpia `\$\_SESSION` y llama a `session\_destroy()`.

8. Seguridad

8.1 Estrategia de seguridad

La estrategia actual es adecuada para entorno de desarrollo y para una aplicación académica/local: autenticación con contraseña hash, sesiones PHP, RBAC por controlador, consultas preparadas, CSRF en formularios y escape de salida HTML.

8.2 Autenticación

- Login por email y contraseña.
- `usuario.password\_hash` almacena hash generado con `password\_hash`.
- `password\_verify` compara contraseña introducida con hash almacenado.
- MFA, JWT y OAuth: No implementado.

8.3 Autorización

- Admin: acceso a panel administrativo, alumnos, grupos, salas y eventos.
- Profesor: acceso a su panel, grupos, alumnos, clases, asistencia y calendario; el admin puede entrar en área profesor.
- Alumno matriculado: acceso a panel, tokens, recuperaciones, calendario y avisos de ausencia.
- Alumno posible/invitado: acceso a reserva/cancelación de clase de prueba.
- ACL granular externa: No implementada; se usa RBAC mediante sesión y validaciones SQL.

8.4 Protección contra ataques

Riesgo	Medida actual
--------	---------------

XSS	Uso de `htmlspecialchars` en salidas dinámicas.
CSRF	`Csrf::token`, `Csrf::validatePost` y campos ocultos en formularios POST.
SQL Injection	Consultas preparadas con `prepare` y `bind_param` en controladores.
Robo de contraseña	Hash de contraseña con API nativa de PHP.
Acceso no autorizado	Validación de rol por controlador y redirección.
Rate limiting	No hay limitador de intentos de login.

9. DevOps e Infraestructura

9.1 Entornos

En este proyecto solo se ha configurado un entorno de desarrollo local. Como mejora futura, podría ampliarse con pruebas automatizadas, un entorno de staging y un despliegue controlado.

9.2 Contenerización

Dockerfile y Docker Compose: No aplica / No implementado. El repositorio está pensado para ejecutarse en XAMPP dentro de `htdocs`.

9.3 Infraestructura cloud

AWS / Azure / GCP: No aplica / No implementado. No hay servicios cloud configurados ni archivos de infraestructura como código.

10. Testing

10.1 Estrategia de pruebas

La estrategia aplicable al estado actual es prueba manual funcional por rol, validando instalación, login, navegación, formularios, persistencia en MySQL y reglas principales de asistencia y tokens.

10.2 Herramientas

Herramienta	Estado
phpMyAdmin	Para verificar tablas, datos demo e importación de SQL.
Navegador	Usado para pruebas funcionales manuales.

10.3 Cobertura

La cobertura se expresa como manual en casos críticos.

10.4 Casos de prueba

ID	Caso	Resultado esperado
TC-01	Ejecutar `setup.php` en XAMPP.	Se crea `punto_de_partida`, tablas, salas, grupos iniciales y usuarios demo.
TC-02	Login admin con `lucia@mail.com` / `1234`.	Redirección a `/admin` y acceso al dashboard.
TC-03	Login profesor con `luis@mail.com` / `1234`.	Redirección a `/profesor` y acceso a clases/asistencia.
TC-04	Login alumno matriculado con `juan@mail.com` / `1234`.	Redirección a `/alumno` con próximas clases y tokens.
TC-05	Login alumno posible con `maria@mail.com` / `1234`.	Redirección a `/invitado`.
TC-06	Crear alumno desde admin.	Alumno guardado y visible en listado correspondiente.
TC-07	Matricular alumno posible.	Estado cambia a `matriculado` y aparece en matriculados.
TC-08	Crear/editar grupo desde admin.	Grupo persiste con sala, profesor, horario, nivel y tipo.
TC-09	Pasar asistencia como profesor.	Se crean/actualizan registros en `asistencia`.
TC-10	Alumno avisa ausencia con más de 24 horas.	Se registra ausencia y se genera token si no supera el máximo de 4.
TC-11	Alumno reserva recuperación.	Se crea `recuperacion` y el token queda usado.
TC-12	Invitado reserva clase de prueba.	Se crea recuperación de prueba si hay token y plazas.
TC-13	Crear evento e inscribir alumno.	Se crea inscripción sin superar aforo.
TC-14	Intentar POST sin CSRF válido.	La acción se rechaza y redirige con mensaje de error.

11. Despliegue

11.1 Procedimiento de despliegue

- Copiar el proyecto dentro de `htdocs/Gestor-Escuela-de-Teatro` o la carpeta equivalente de `htdocs`.
- Iniciar Apache y MySQL desde XAMPP.
- Revisar credenciales en `config/database.php` si el MySQL local no usa `root` sin contraseña.

- Abrir `http://localhost/Gestor-Escuela-de-Teatro/setup.php` para crear base de datos y datos iniciales.
- Abrir `http://localhost/Gestor-Escuela-de-Teatro/` y entrar con usuarios demo.
- Opcional: importar scripts de `sql/` desde phpMyAdmin para datos de prueba adicionales.

Build: No aplica. No existe paso de compilación, gestor de paquetes frontend ni Composer.

Variables de entorno: No implementadas; la configuración de BD está en PHP.

11.2 Rollback

- Código: volver a una versión anterior del repositorio mediante Git.
- Base de datos: restaurar un backup/exportación SQL previa desde phpMyAdmin o MySQL CLI.
- Datos demo: volver a ejecutar `setup.php` o reimportar scripts SQL controlados cuando proceda.

11.3 Configuración de servidores

Apache/XAMPP, servidor previsto para desarrollo local.

12. Mantenimiento y Soporte

- Mantener actualizado `README.md` cuando cambien rutas, usuarios demo, instalación o scripts.
- Antes de cambios de esquema, exportar la base de datos y crear migración SQL en `database/migrations` o `sql`.
- Revisar periódicamente el uso de CDNs externos y fijar versiones cuando sea necesario.
- Para incidencias funcionales, reproducir primero con usuarios demo y verificar tablas implicadas en MySQL.
- Para problemas de acceso, comprobar sesión, rol en `usuario`, estado en `alumno` y redirecciones del controlador.

13. Buenas Prácticas

13.1 Estándares de código

- Mantener separación entre rutas, controladores, vistas y configuración.
- Usar consultas preparadas para toda entrada externa.

- Escapar salidas dinámicas con ``htmlspecialchars``.
- Validar CSRF en acciones POST que modifican datos.
- Evitar duplicar rutas y centralizar mensajes frecuentes cuando se amplíe el proyecto.

13.2 Git, GitHub y control de versiones

- Usar ramas descriptivas para cambios funcionales o correcciones.
- Hacer commits pequeños y trazables, por ejemplo ``docs: completar documentación técnica`` o ``fix: validar csrf en formulario``.
- No versionar credenciales reales ni dumps con datos sensibles.
- Revisar ``git status`` antes de entregar para separar cambios de documentación de cambios de código.

13.3 Documentación

- ``README.md``: mantener como guía corta de instalación y uso.
- Esta documentación técnica: actualizar cuando cambie arquitectura, base de datos, rutas o seguridad.

14. Anexos

14.1 Scripts útiles

Archivo	Uso
<code>`setup.php`</code>	Crea base de datos, tablas, salas, grupos iniciales y usuarios demo.
<code>`sql/datos_prueba_alumno.sql`</code>	Datos adicionales para probar flujos de alumno.
<code>`sql/datos_invitado.sql`</code>	Datos para probar clase de prueba/invitado.
<code>`sql/30_alumnos_demo.sql`</code>	Carga de alumnos demo para listados y pruebas de volumen básico.
<code>`sql/generar_clases_junio_2026.sql`</code>	Generación de clases de prueba para junio de 2026.
<code>`sql/limpiar_clases_duplicadas.sql`</code>	Limpieza de clases duplicadas y restricción única.
<code>`database/migrations/2026_04_30_observaciones.sql`</code>	Migración relacionada con observaciones.
<code>`database/migrations/2026_05_04_observacion_profesor.sql`</code>	Crea/ajusta <code>`observacion_profesor`</code> .

14.2 Usuarios demo

Rol	Email	Contraseña	Ruta esperada
Admin	<code>`lucia@mail.com`</code>	<code>`1234`</code>	<code>`/admin`</code>

Profesor	`luis@mail.com`	`1234`	`/profesor`
Alumno matriculado	`juan@mail.com`	`1234`	`/alumno`
Alumno posible/invitado	`maria@mail.com`	`1234`	`/invitado`

14.3 Referencias técnicas

- PHP `password\_hash` y `password\_verify` para contraseñas.
- PHP `mysqli` con consultas preparadas.
- MySQL InnoDB con claves foráneas, claves únicas y `utf8mb4`.
- SweetAlert2 y Font Awesome cargados por CDN en vistas concretas.